



Linearity and Uniqueness: An Entente Cordiale

Danielle Marshall¹(✉) , Michael Vollmer¹ , and Dominic Orchard^{1,2}

¹ University of Kent, Canterbury, UK
{dm635,m.vollmer,d.a.orchard}@kent.ac.uk

² University of Cambridge, UK

Abstract. Substructural type systems are growing in popularity because they allow for a resourceful interpretation of data which can be used to rule out various software bugs. Indeed, substructurality is finally taking hold in modern programming; Haskell now has linear types roughly based on Girard’s linear logic but integrated via graded function arrows, Clean has uniqueness types designed to ensure that values have at most a single reference to them, and Rust has an intricate ownership system for guaranteeing memory safety. But despite this broad range of resourceful type systems, there is comparatively little understanding of their relative strengths and weaknesses or whether their underlying frameworks can be unified. There is often confusion about whether linearity and uniqueness are essentially the same, or are instead ‘dual’ to one another, or somewhere in between. This paper formalises the relationship between these two well-studied but rarely contrasted ideas, building on two distinct bodies of literature, showing that it is possible and advantageous to have both linear and unique types in the same type system. We study the guarantees of the resulting system and provide a practical implementation in the graded modal setting of the Granule language, adding a third kind of modality alongside coeffect and effect modalities. We then demonstrate via a benchmark that our implementation benefits from expected efficiency gains enabled by adding uniqueness to a language that already has a linear basis.

Keywords: linear types · uniqueness types · substructural logic

1 Introduction

Linear types [15, 57] and uniqueness types [5, 47] are two influential and long-standing flavours of substructural type system. As these approaches have developed, it has become clear in the community (both in folklore and the literature) that these are closely related ideas. For example, the chapter on substructurality in *Advanced Topics in Types and Programming Languages* [62] describes uniqueness types as “a variant of linear types”. This framing is supported by various works which, for example, make reference to “a form of linearity (called uniqueness)” [33] or other such statements of equality or similarity [38].

But reading a different set of papers gives a contrasting impression that linearity and uniqueness are not the same but in some sense *dual* to one another,

and with different behaviour for at least some applications. Recent work on linear types for Haskell [7] describes the two concepts as being “at their core, dual” and later having a “weak duality”. The impression that these two approaches behave differently is backed up by much of the theoretical work on uniqueness types, with one paper stating that “although both linear logic and uniqueness typing are substructural logics, there are important differences” [56], closely followed by a tantalising mention of the fact that “some systems based on linear logic are much closer to uniqueness typing than to linear logic”.

It is clear, at least, that both linear types and uniqueness types are *substructural* type systems: they both restrict structural rules (in particular, contraction and weakening) of type systems that are the Curry-Howard counterparts to regular intuitionistic logic. This captures the well-known maxim that “not all things in life are free” [61]; many kinds of data behave *resourcefully*, and are subject to constraints on their usage. Sensitive data should not be infinitely duplicated and passed around freely, file handles should not be arbitrarily discarded without being properly closed, and communication channels should not be used without adherence to a fixed protocol, to name a few!

Thanks to these clear benefits, notions of substructurality are slowly but surely making their way into the programming ecosystem, with languages such as Haskell [7], Idris [9], Clean [47], Rust [24], ATS [65], and Granule [36] all having type systems that behave substructurally in some way. What is not clear, however, is what exactly the relationship is between these varying systems; for instance, it is not obvious how to relate linearity and uniqueness. Linear types, though they themselves come in various forms, are most often based on the linear logic of Girard [15], and in the strictest sense they treat values as resources which must be used exactly once and never again. On the other hand, uniqueness types are named as such because they aim to ensure that values are guaranteed to have at most one reference to them [40, 45, 47, 48, 55, 56], with a view towards allowing them to be safely updated in-place. Do these two requirements always coincide, or are there cases where they diverge?

In this paper, we resolve this long-standing confusion, building on two distinct bodies of literature to develop an accurate understanding of the contexts in which linear and uniqueness types behave the same or behave differently, and their relative strengths and weaknesses. Our primary contributions are as follows:

- In Section 2, we discuss the contrasting understandings of the relationship between linearity and uniqueness, and draw together aspects of these viewpoints to intuitively describe the link between these concepts.
- In Section 3, we formalise these notions by developing a unified calculus and type system that incorporates linear, unique, and Cartesian (unrestricted or non-unique/non-linear) types all at once, building on the linear λ -calculus. We give an operational model via a heap semantics which allows us to prove various key operational guarantees for both linearity and uniqueness.
- In Section 4, as a proof of concept, we implement uniqueness types into the language Granule which already has a linear basis, introducing a third flavour of modality alongside the graded comonadic (coeffectful) and graded

monadic (effectful) modalities already present in the language. The implementation enables the classic primary use of uniqueness: access to safe in-place update in a functional language without working inside a monad.

- In Section 4.2, we confirm the performance benefits of uniqueness types by benchmarking the performance of arrays which allow for in-place update. We generate impure Haskell code from our Granule implementation in order to demonstrate that further efficiency can be gained via adding uniqueness types even when your language is already linear at its core.

Section 5 and Section 6 provide related work and discussion, including relation to ideas in Rust. Various additional details are collected in the appendix [28], including proofs and collected reduction rules for the operational semantics. We also provide an artifact [29], so that the interested reader can experiment with code examples in Granule and reproduce our benchmarks for themselves.

2 Key Ideas

It is clear that linear and uniqueness types both involve restricting the substructural rules of intuitionistic logic, but what remains unclear is the exact relationship between the two concepts. This section discusses two widespread understandings of their relationship, both of which are accurate in some respects but fail to capture some key similarities and differences. We then combine aspects of both viewpoints to systematically relate linearity and uniqueness.

2.1 Are linearity and uniqueness (essentially) the same?

Perhaps the most well-known substructural types are *linear* types, which have been studied for decades in the literature [57, 62] as the Curry-Howard counterpart of linear logic [15]. Several languages have implemented linear type systems over the years, including ATS [65], Alms [52] and Quill [32], and they are steadily making their way into the mainstream via extensions to languages like Haskell [7]. Examples of linearity in this paper will focus on the functional language Granule [36] (whose syntax resembles Haskell), since values in Granule are linear by default making the examples less complex, and also because Granule will later be the foundation upon which we build our unified calculus.

Strictly, linear types treat values as resources which must be used once and then never again. For instance, we can type the identity function, since it binds a single variable and then uses it, but the K combinator (which discards one of its arguments) is not linearly typed. Thus linearity is a claim about the consumption of a resource: a linear type is a contract, which says that we must consume a value that we are given exactly once. Consider the following classic example (rendered in Granule) of a function which cannot be represented using linear types, assuming an interface where `eat : Cake → Happy` and `have : Cake → Cake`:

```
1 impossible : Cake → (Happy, Cake)
2 impossible cake = (eat cake, have cake)
```

— Ill-typed Granule —

Note that Granule’s function type $\rightarrow$ is the type of linear functions, more traditionally written $\multimap$. The above function is ill-typed and the Granule compiler will brand it with a linearity error; this is because the value of type `Cake` passed into the function is a linear resource, and the body of the function requires us to duplicate it (via contraction), which is forbidden. Thus, linear types remind us of the familiar aphorism: *you can’t have your cake and eat it too*.

Uniqueness types, on the other hand, are primarily aimed at ensuring that values have only a single reference to them, which is a useful property for ensuring the safety of updating data in-place. But is this uniqueness restriction really so different from the constraints of linearity?

One of the most familiar languages featuring uniqueness types is Clean [47], which uses uniqueness for mutable state and input/output, in contrast to languages such as Haskell which use monads for similar purposes. We shall use Clean for our uniqueness examples for the moment, before we introduce our own implementation of uniqueness in Section 4. Consider the following in Clean:

```
1 impossible :: *Coffee -> (*Awake, *Coffee)
2 impossible coffee = (drink coffee, keep coffee)
```

— Ill-typed Clean —

We use coffee instead of cake to distinguish unique values from the linear values of the Granule example, but notice this function has exactly the same structure as the previous example. The operator `*` denotes a unique type, since unrestricted values are the default in Clean. Similarly to Granule, when presented with this function Clean gives a uniqueness error; the argument of type `*Coffee` is duplicated, and so we can no longer guarantee there is only one reference to it upon exiting the function. Think of a `*Coffee` as having been freshly poured; we cannot continue acting as though it is fresh once some of it has been drunk!

So far, it seems that the concepts of linearity and uniqueness are very similar after all, as is often claimed. However, neither of these examples uses *unrestricted* values; we only see values that are linearly typed or uniquely typed. In fact, in a setting where all values must be linear, we can also guarantee that every value is unique, and vice versa! Intuitively, if it is never possible to duplicate a value, then it will never be possible for said value to have multiple references. It is when we also have the ability for unrestricted use (non-linear/non-unique) that differences between linearity and uniqueness begin to arise, as we will soon see.

Much of the classic literature on linear types makes mention of the idea that linearity can be used for tracking whether a value has only one reference, though we know by now that this more accurately describes *uniqueness*; indeed, one of the oldest such papers by Wadler, which has been (rightly) hugely influential, states that “values of linear type have exactly one reference to them, and so require no garbage collection” [57, p.2]. However, systems akin to the one discussed by Wadler [4, 18, 46] crucially separate values into two completely distinct and (mostly) disconnected linear and non-linear worlds. In this context, a linear value can never have been duplicated previously and thus must also obey the

conditions required for uniqueness. Therefore, it is correct to say that a value of linear type has exactly one reference in such a system.

This issue is further discussed in a later article by Wadler [58], though uniqueness types had yet to be invented and so the concept is never referred to by this name. Linear types based on linear logic are defined in Section 3 of said article, for which linearity behaves as we understand it: a value having linear type guarantees that it will not be duplicated or discarded, but the notion of *dereliction* allows a non-linear variable to be used linearly going forwards [15]. As Wadler states, “dereliction means we cannot guarantee *a priori* that a variable of linear type has exactly one pointer to it” [58, p.7], and so we cannot guarantee uniqueness of reference in a system based upon linear logic. In Section 7, Wadler goes on to define *steadfast* types, where dereliction and promotion are again restricted to recover the uniqueness guarantee in addition to the linearity restriction ³.

However, never being able to duplicate or discard any value is an overly restrictive view of data, preventing many valid uses of various kinds of information, and so modern languages with linear types therefore generally do provide a mechanism for non-linearity rather than working in the ‘steadfast’ style. Linear logic provides the **!** modality (also called the *exponential* modality), which allows the representation of non-linear (unrestricted) values. In Granule, we can use this modality to rewrite the previous example into one that is now well-typed:

```
1 possible : !Cake → (Happy, Cake)
2 possible lots = let !cake = lots in (eat cake, have cake)
```

Granule

We can think of **!Cake** values as representing an infinite amount of cake, which is made available once we eliminate the modality (via the **let**) to get an unrestricted (non-linear) variable **cake**. The functions **eat** and **have** are linear functions, so each application in isolation views **cake** linearly, by an implicit use of dereliction in the type system. Crucially, from an unrestricted value we can produce a linear value, so we can impose the restriction of linearity whenever we like. However it is not possible to produce an unrestricted value from a linear one. This restriction means that linear types are useful for representing resources such as file handles, as in the following example:

```
1 twoChars : (Char, Char) <IO>
2 twoChars = let                                     -- do-notation like syntax
3     h      ← openHandle ReadMode "someFile";
4     (h, c1) ← readChar h;
5     (h, c2) ← readChar h;
6     ()      ← closeHandle h
7     in pure (c1, c2)
```

Granule

³ The concept of steadfastness coincides with the notion of “necessarily unique” used in languages such as Clean, where a necessarily unique value is one that is unique and also can never be made non-unique [47].

Here, we open a file handle, read two characters from it, and then close it. The linearity of the handle ensures that once we have created it, we must close it properly, and also that we cannot duplicate it along the way. But linearity is less useful in other circumstances. As an example, consider the case of mutable arrays. Discarding an array will not cause any problems,⁴ so a linear array would be too restrictive and not allow for some valid use cases; *affine types* allow discarding behaviour by adding back in weakening [52]. But in order to be able to mutate an array we need to be able to guarantee that no other references to it exist, and in this sense linearity is not strong enough; any linear value could have previously been a non-linear one that was duplicated any number of times before being specialised (via dereliction) to a linear type. For representing mutable arrays, we are better served by considering uniqueness types.

Uniqueness behaves differently to linearity in the context of a system with the ability to describe unrestricted values. If we have an unrestricted value, we certainly cannot produce a unique one from it which would violate the guarantee of uniqueness; we cannot claim that a value has only one reference to it when it could have been duplicated and manipulated elsewhere. But conversely, if we have a unique value, there is no harm in dropping this guarantee and producing an unrestricted value; a non-unique value does not need to make any promises about how many references may exist. Thus, in Clean we can write:

```
1 possible :: *Coffee -> (Awake, Coffee)
2 possible coffee = (drink coffee, keep coffee)
```

———— Clean

Here, we require that the input is unique (it has type **Coffee*), so for the function to be well-typed we can no longer claim this value is unique once it reaches the output, as it has been duplicated along the way (and it now must have type *Coffee*). The information here is flowing in the opposite direction than for linearity; the *possible* function in Clean would be ill-typed if we replaced unique values with linear ones, and vice versa for the earlier Granule example. This directionality allows us to represent mutable arrays much more easily with uniqueness. For example, the following destructively fills a real-valued array:

```
1 fill :: *{Real} Int -> *{Real}
2 fill a1 0 = a1
3 fill a1 i
4     # f = toReal i
5     # a2 = {a1 & [i - 1] = f}    // write f to index i-1 in a1
6     = fill a2 (i - 1)           // recurse with unique array a2
```

———— Clean

⁴ One may worry that discarding an array could cause space leaks, but this can be tempered via garbage collection. If a non-linear value will no longer be used we know statically that it can be garbage collected, and thus it is harmless to reuse the space occupied by this value going forwards. This will allow us to update unique objects such as arrays destructively without being concerned about referential transparency.

Here, we take in a unique array of floating point numbers and some unrestricted integer value, and fill the first cells of the array with the numbers up to that value. Here we know that it is safe to write to the array because it is unique, so no other references to it can exist elsewhere; once we are finished with the array later on, however, it is fine to discard it, as with an array in most other functional programming languages, which would not be possible if our array was linearly typed. This however does mean that uniqueness types are not appropriate for the earlier example of file handles—we cannot ensure that a unique file handle is closed, as it can be discarded at any time.

In summary, linearity and uniqueness provide the same guarantees up until a system also has a notion of unrestricted value (non-linear or non-unique). The complementary but distinct use cases shown above make it clear that it would be valuable to have both linear and unique values together in a single language, but this has previously not been possible. Our main contribution is a core calculus that allows linearity and uniqueness to coexist and interact, demonstrated also via an implementation in the Granule language. Next we consider the question of duality, and how to formally describe how linearity and uniqueness differ.

2.2 Are linearity and uniqueness dual?

It is common in folklore and in the literature to describe linearity and uniqueness as somehow dual to one another (see e.g., [32, 52]) but rigorous versions of this statement are more rarely found. The earliest formalisation of uniqueness is from Harrington’s ‘uniqueness logic’ [20], which we use as a foundation for much of the following. Harrington constructs a logic which is on the surface much like linear logic, but instead of the $!$ modality for non-linearity it includes a $\circ$ modality for non-uniqueness which differs from non-linearity in its introduction rule.

In linear logic, the $!$ modality acts as a comonad, such that the introduction of $!$ on the right of a sequent means that all formulae on the left of a sequent must also have $!$ applied, whilst introduction of $!$ on the left is unrestricted:

$$\frac{! \Gamma \vdash P}{! \Gamma \vdash ! P} !_R \quad \frac{\Gamma, P \vdash Q}{\Gamma, ! P \vdash Q} !_L$$

(also known as *storage* and *dereliction* respectively [16]). In contrast, the non-uniqueness modality $\circ$ of Harrington acts as a monad, meaning that introduction of $\circ$ on the right is unrestricted but introduction of $\circ$ on the left of a sequent means that all formulae on the right of the sequent must also have $\circ$ applied:

$$\frac{\Gamma \vdash P}{\Gamma \vdash P^\circ} \circ_R \quad \frac{\Gamma, P \vdash Q^\circ}{\Gamma, P^\circ \vdash Q^\circ} \circ_L$$

The non-uniqueness modality $\circ$ then has the following structural rules for contraction and weakening, which are conspicuously identical to those for the $!$ modality representing non-linearity:

$$\frac{\Gamma, P^\circ, P^\circ \vdash R}{\Gamma, P^\circ \vdash R} \circ_C \quad \frac{\Gamma \vdash R}{\Gamma, P^\circ \vdash R} \circ_W \quad \frac{\Gamma, ! P, ! P \vdash R}{\Gamma, ! P \vdash R} !_C \quad \frac{\Gamma \vdash R}{\Gamma, ! P \vdash R} !_W$$

One might be tempted to think that because the introduction rules for $\circ$ behave dually to those of $!$, the modalities are simply dual to one another, and thus non-uniqueness is equivalent to linear logic's $?$. But since the contraction and weakening rules for $\circ$ are the same as those for $!$, this is not quite the case; $\circ$ behaves dually to $!$ in some ways but not in others. Formally, $\circ$ is a monad while $!$ is a comonad, but both are comonoidal, whereas $?$ is monoidal.

Linear logic allows us to derive $!P \vdash P$ (from dereliction), which agrees with our notion of linearity where non-linear values can be restricted to behave linearly going forwards but if we have a linear value it must remain linear; uniqueness logic conversely allows us to derive $P \vdash P^\circ$, formalising our concept of uniqueness where we can forget the uniqueness guarantee and turn a unique value into a non-unique one, but if we have a non-unique value we cannot go back.

We can now make more precise the intuitive notion we have developed, which suggests that linear types provide a restriction on what can be done with a value 'in the future' whilst uniqueness types provide a guarantee about what has been done with a value 'in the past'. The distinction becomes clearer when we consider substitutions, which are generated by β -reductions.

Substitutions are the same whether we are working with linear logic or uniqueness logic, as the rules for functions are identical, but the difference arises when thinking about what it is possible to know about a value in one logic compared to the other. Given a linear value, we know that substituting this value into an expression will preserve linearity, as there is no way to transform a linear value into a non-linear one. Conversely, given a unique expression then we know that any values substituted in will not affect the uniqueness guarantee, as there is no way to transform a non-unique value into a unique one. Thus 'future' refers to outgoing substitutions, while 'past' refers to incoming substitutions.

So if linearity and uniqueness do in fact behave the same in some ways but not all, and they do in fact behave dually in some ways but not all, then what is the overall takeaway? What statement can we make about the relationship between their behaviour that reconciles these two viewpoints?

Takeaway. *Linearity and uniqueness behave dually with respect to composition, but identically with respect to structural rules, i.e., their internal plumbing.*

In other words, internally the non-linear and non-unique modalities are both comonoidal, so they allow for the same behaviour of contraction and weakening for values that are wrapped inside them.

But the duality arises upon considering how we can map into and out of these modalities; we can map out of the non-linear modality and retrieve a linear value, but we can never map into it, giving the modality its familiar comonadic structure. Conversely, we can map a unique value into the non-unique modality to allow for contraction and weakening, but we can never map back out of it, which explains the dual monadic behaviour of this modality.

It is this understanding of the similarities and differences between linearity and uniqueness that will allow us to unify them, and have values of both flavours present in a single type system, which will be our goal for the next section.

3 The Linear-Cartesian-Unique Calculus

We now consider how to represent both linearity and uniqueness in the same system. The first choice to make is whether our base values will be linear or unique⁵, as this will influence the directionality of the modalities we need to include in the calculus. Here we present a system where linearity is the base and uniqueness is a modality, as opposed to one where uniqueness is the base and linearity is a modality, for two reasons.

- The first reason is pragmatic; we later implement our approach in Granule, which already has linear values as the default. Therefore, including uniqueness as an additional modality in the system will require far fewer changes to the language, since a unique base would most likely require a redesign. Moreover, languages with uniqueness types like Clean generally have non-unique values as their default, with uniqueness having to be specifically annotated; a system with a uniqueness modality will also map more closely onto these languages than one where uniqueness is the basis.
- The second reason is that developing a sound calculus with a unique base is more complex. Consider such a hypothetical calculus with a modality $\circ$ representing unrestricted values and a modality $\bullet$ representing linear values. If we construct a product of linear values $(a^\bullet, b^\bullet)$, then this product is unique (rather than linear), so we can promote to an unrestricted product $(a^\bullet, b^\bullet)^\circ$ and freely duplicate the product, though the values contained within are linear. A linear base avoids this problem (among others) as products being linear by default means that their usage is maximally restricted, so there is no circumventing either a uniqueness guarantee via their construction or a linearity restriction via their duplication.⁶

Given a linear basis, we formalise the idea that we can map from unique to non-unique and from non-linear to linear. The key insight is that *we treat non-linearity and non-uniqueness as the same state* as both these states are unrestricted; we can do anything we like with, and have no guarantees for, an unrestricted value. We write $*P$ for a P with a uniqueness guarantee, similar to the syntax of Clean and to avoid confusion with Harrington’s $\circ$ modality for non-uniqueness. The resulting calculus, which we call the Linear-Cartesian-Unique calculus (or LCU for short), builds on (intuitionistic multiplicative exponential) linear logic with additional rules for uniqueness.

⁵ We choose a substructural basis over an unrestricted one since this more closely maps to both linear and uniqueness logic, where values have substructural behaviour by default unless they are wrapped in a modality.

⁶ A similar problem arises from the application of unique functions, and this has been a thorn in the side of developers of uniqueness type systems for some time. The solution applied in Clean is that any function with unique elements in its closure is “necessarily unique”, meaning it cannot be subtyped into a non-unique function and applied multiple times. Handily, this coincides with the notion of a linear function, which is why our calculus having a linear base also avoids this problem.

Syntax LCU's syntax is that of the linear λ -calculus with multiplicative products and unit (first line of syntax below) with terms for introducing and eliminating the $!$ modality and working with the uniqueness modality (second line):

$$\begin{aligned} t ::= & x \mid \lambda x. t \mid t_1 t_2 \mid (t_1, t_2) \mid \text{let } (x, y) = t_1 \text{ in } t_2 \mid \text{unit} \mid \text{let unit} = t_1 \text{ in } t_2 \\ & \mid !t \mid \text{let } !x = t_1 \text{ in } t_2 \mid \&t \mid \text{copy } t_1 \text{ as } x \text{ in } t_2 \mid *t \end{aligned} \quad (\text{terms})$$

The meaning is explained in the next section with reference to typing.

3.1 Typing

Typing judgments are of the form $\Gamma \vdash t : A$, with types A defined:

$$A, B ::= A \multimap B \mid A \otimes B \mid 1 \mid !A \mid *A \quad (\text{types})$$

Thus our type syntax comprises linear function types $A \multimap B$, linear multiplicative products $A \otimes B$, a linear multiplicative unit 1 , the non-linearity modality $!A$ and the uniqueness modality $*A$.

Typing contexts are defined as follows:

$$\Gamma ::= \emptyset \mid \Gamma, x : A \mid \Gamma, x : [A] \quad (\text{contexts})$$

which are either empty, or contexts extended with a linear assignment $x : A$ or contexts extended with a non-linear assignment denoted $x : [A]$. This marking of assumptions in a context as linear or non-linear (see Terui [50]) is one way to guarantee substitution is admissible (avoiding, for example, issues pointed out by Wadler where substitution is not well-typed if care is not taken [59, 60], an issue noted also by Prawitz in 1965 in the context of S4 modal logic [42]).

Throughout, the comma operator $,$ concatenates disjoint contexts.

We introduce the key typing rules inline. Figure 1 collects the full set of rules. The linear λ -calculus core is typed by the following three rules:

$$\frac{}{[\Gamma], x : A \vdash x : A} \text{VAR} \quad \frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x. t : A \multimap B} \text{ABS} \quad \frac{\Gamma_1 \vdash t_1 : A \multimap B \quad \Gamma_2 \vdash t_2 : A}{\Gamma_1 + \Gamma_2 \vdash t_1 t_2 : B} \text{APP}$$

In the case of VAR, a linear variable is used but the rest of the context must be marked as non-linear, denoted by $[\Gamma]$ which marks all assumptions as non-linear.

Definition 1 (All non-linear assumptions). A context Γ is denoted as containing only non-linear assumptions by writing $[\Gamma]$ in the typing rules, where $[\emptyset]$ and $[\Gamma] \Rightarrow [\Gamma], x : [A]$.

In the case of APP, the two subterms are typed in different contexts which are then combined via *context addition*.

Definition 2 (Context addition). The partial operation $+$ on contexts is the union of two contexts as long as they are disjoint in their linear assumptions and any variables occurring in both contexts are both non-linear assumptions, i.e.

$$\Gamma_1 + \Gamma_2 = \Gamma_1 \cup \Gamma_2 \quad \text{iff} \quad \forall x \in \text{dom}(\Gamma_1) \cap \text{dom}(\Gamma_2) \Rightarrow \exists A. \Gamma_1(x) = \Gamma_2(x) = [A]$$

The non-linear modality $!$ has the following introduction and elimination rules and related dereliction rule:

$$\frac{[\Gamma] \vdash t : A}{[\Gamma] \vdash !t : !A} !_I \quad \frac{\Gamma_1 \vdash t_1 : !A \quad \Gamma_2, x : [A] \vdash t_2 : B}{\Gamma_1 + \Gamma_2 \vdash \text{let } !x = t_1 \text{ in } t_2 : B} !_E \quad \frac{\Gamma, x : A \vdash t : B}{\Gamma, x : [A] \vdash t : B} \text{DER}$$

The left-most rule captures the idea that a computation t of value A can be used non-linearly, by ‘promoting’ it to $!A$ as long as all its inputs are also non-linear, denoted by $[\Gamma]$ in the context. The middle rule eliminates a non-linear modality (a capability to use an A value non-linearly) by composing it with a variable x which is non-linear in t_2 . These rules are accompanied by the ‘dereliction’ rule that says non-linear variables can be treated as linear variables.

So far everything is standard from other linear type systems. We now move to our uniqueness modality which has two syntactic constructs: *borrow* and *copy*:

$$\frac{\Gamma \vdash t : *A}{\Gamma \vdash \&t : !A} \text{BORROW} \quad \frac{\Gamma_1 \vdash t_1 : !A \quad \Gamma_2, x : *A \vdash t_2 : !B}{\Gamma_1 + \Gamma_2 \vdash \text{copy } t_1 \text{ as } x \text{ in } t_2 : !B} \text{COPY}$$

The borrow rule maps a unique value to a non-linear value, allowing a uniqueness guarantee to be forgotten. In terms of the operational semantics (see Section 3.4), this causes evaluation of t before the borrow. Next, the copy rule says that a non-linear value of type A can be copied to produce a unique A which is used by t_2 ; the input is required to be non-linear so that we cannot circumvent a linearity restriction by copying a linear value, and the output is required to be non-unique so that we cannot leverage the copy to smuggle out a value which pretends to be truly unique. These rules in turn are accompanied by the ‘necessitation’ rule that says values can be assumed unique as long as they have no dependencies:

$$\frac{\emptyset \vdash t : A}{[\Gamma] \vdash *t : *A} \text{NEC}$$

The borrow and copy rules in this logic suggest a monad-like relationship between the $!$ and $*$ modalities, with the borrow rule representing the ‘return’ of the monad and the copy rule likewise acting as the ‘bind’. The $*$ modality is not in itself a monad (or indeed, a comonad like $!$); rather, it acts as a functor over which the $!$ modality becomes a *relative monad* [3]. A relative monad comprises a functor J and an object mapping T , along with an operation $\eta : JX \rightarrow TX$ and a mapping from $JX \rightarrow TY$ arrows to $TX \rightarrow TY$ with axioms analogous to the monad axioms. Thus, here J is the uniqueness modality $*$ and T the non-linearity modality $!$. If one imagines the dual version of this logic where the basis is unique, the hypothetical linearity modality would act as a functor making the non-uniqueness modality into a *relative comonad* [1, 37] in much the same way.

3.2 Equational theory

One way of understanding the meaning of the LCU calculus is to see its equational theory (which we later prove sound against its operational model). The

$\overline{[T], x : A \vdash x : A}$	VAR	$\frac{\Gamma, x : A \vdash t : B}{\Gamma \vdash \lambda x. t : A \multimap B}$	ABS	$\frac{\Gamma_1 \vdash t_1 : A \multimap B \quad \Gamma_2 \vdash t_2 : A}{\Gamma_1 + \Gamma_2 \vdash t_1 t_2 : B}$	APP
$\frac{\Gamma_1 \vdash t_1 : A \quad \Gamma_2 \vdash t_2 : B}{\Gamma_1 + \Gamma_2 \vdash (t_1, t_2) : A \otimes B}$		$\otimes_I$	$\frac{\Gamma_1 \vdash t_1 : A \otimes B \quad \Gamma_2, x : A, y : B \vdash t_2 : C}{\Gamma_1 + \Gamma_2 \vdash \text{let } (x, y) = t_1 \text{ in } t_2 : C}$		$\otimes_E$
$\overline{[T] \vdash \text{unit} : 1}$		1_I	$\frac{\Gamma_1 \vdash t_1 : 1 \quad \Gamma_2 \vdash t_2 : B}{\Gamma_1 + \Gamma_2 \vdash \text{let unit} = t_1 \text{ in } t_2 : B}$		1_E
$\frac{\Gamma, x : A \vdash t : B}{\Gamma, x : [A] \vdash t : B}$	DER	$\frac{[T] \vdash t : A}{[T] \vdash !t : !A}$	$!_I$	$\frac{\Gamma_1 \vdash t_1 : !A \quad \Gamma_2, x : [A] \vdash t_2 : B}{\Gamma_1 + \Gamma_2 \vdash \text{let } !x = t_1 \text{ in } t_2 : B}$	$!_E$
$\frac{\Gamma \vdash t : *A}{\Gamma \vdash \&t : !A}$	BORROW	$\frac{\Gamma_1 \vdash t_1 : !A \quad \Gamma_2, x : *A \vdash t_2 : !B}{\Gamma_1 + \Gamma_2 \vdash \text{copy } t_1 \text{ as } x \text{ in } t_2 : !B}$	COPY	$\frac{\emptyset \vdash t : A}{[T] \vdash *t : *A}$	NEC

Fig. 1: Collected typing rules for LCU calculus

calculus has the standard $\beta\eta$ -equalities for the multiplicative linear λ -calculus fragment, which includes the following $\beta\eta$ rules for $!$:

$$\text{let } !x = !t \text{ in } t' \equiv [t/x]t' \quad (\beta!)$$

$$\text{let } !x = t \text{ in } !x \equiv t \quad (\eta!)$$

along with the following equalities on the uniqueness fragment:

$$\text{copy } t \text{ as } x \text{ in } \&x \equiv t \quad (\text{unitR})$$

$$\text{copy } \&v \text{ as } x \text{ in } t' \equiv [v/x]t' \quad (\text{unitL})$$

$$\text{copy } t_1 \text{ as } x \text{ in } (\text{copy } t_2 \text{ as } y \text{ in } t_3) \equiv \text{copy } (\text{copy } t_1 \text{ as } x \text{ in } t_2) \text{ as } y \text{ in } t_3 \quad (\text{assoc})$$

The first axiom states that copying a non-linear t into a unique value x and immediately borrowing it to be non-linear is equivalent to just t . The second axiom states that borrowing a unique value v and copying it to a unique x in the scope of t' is the same as just substituting in that v for x . The last axiom gives associativity of copying under the side condition that x is free in t_3 . These equations are exactly the relative monad axioms [3], though we specialise (unitL) slightly by restricting to values to account for the reduction semantics.

The typability of these axioms relies on the admissibility of linear and non-linear substitution shown in Section 3.5 on the metatheory of the calculus.

3.3 Exploiting uniqueness for mutation

A key use for ensuring uniqueness of a reference is that this allows mutation to be used safely—the original pun behind Wadler’s “Linear Types can Change the World” [57]. To illustrate this idea, and consider its soundness in the next section, we extend the LCU calculus with a primitive type of arrays:

$$A ::= \dots \mid \text{Array } A \mid \mathbb{N} \mid \mathbb{F}$$

where $\mathbb{N}$ are natural numbers used for sizes and indices and $\mathbb{F}$ floating-point values. The calculus is also extended with operations for floating-point arrays, typed by axiomatic rules (with built-in weakening):

$$\begin{aligned} [I] \vdash \text{newArray} & : \mathbb{N} \multimap * (\text{Array } \mathbb{F}) \\ [I] \vdash \text{readArray} & : * (\text{Array } \mathbb{F}) \multimap \mathbb{N} \multimap \mathbb{F} \otimes * (\text{Array } \mathbb{F}) \\ [I] \vdash \text{writeArray} & : * (\text{Array } \mathbb{F}) \multimap \mathbb{N} \multimap \mathbb{F} \multimap * (\text{Array } \mathbb{F}) \\ [I] \vdash \text{deleteArray} & : * (\text{Array } \mathbb{F}) \multimap 1 \end{aligned}$$

These operations provide the interface for exploiting unique array references, where `writeArray` performs mutation as the type system guarantees that uniquely typed values have not been duplicated in the past (Section 3.5). We ignore out-of-bounds exceptions as this is an orthogonal issue, which could be solved using indexed types. We elide rules for typing numerical terms here.

Our implementation in Section 4 replays these ideas in a practical setting. The next section gives the operational heap model for the calculus, where the semantics of mutation is made concrete.

3.4 Operational heap model

We define an operational model for the LCU calculus to make the meaning of uniqueness and linearity more concrete, and to prove that our type system enforces the desired properties. The semantics is call-by-name and resembles a small-step operational semantics but instead uses a notion of *heaps*, both to capture the idea of a memory reference to arrays as well as to give a way to track resource usage on program variables. We adapt the model of Choudhury et al. [11], which was used to track resource usage in a pure language with graded types. Our model applies this idea to a non-graded setting, extended to include reference counting for uniqueness. To prove that linearity and uniqueness are respected (soundness), the heap semantics incorporates some typing information in order to ease the theorem statements and proofs as shown in Section 3.5.

Single-step reductions in the operational model are of the form:

$$H \vdash t \rightsquigarrow H' \vdash t' \mid \Gamma \mid \Delta \quad (\text{single-step judgment form})$$

where H is the incoming heap which provides bindings to variables that appear in t and array allocations. The result of the reduction is a new term t' with an updated heap H' , as well as two additional pieces of information: Γ gives us a ‘binding context’ recording the typing of any binders that were encountered (or ‘opened’) during reduction, and Δ gives us a ‘usage context’ containing an account of how variables were used. Usage contexts are defined as:

$$\Delta ::= \emptyset \mid \Delta, x : r \quad (\text{usage contexts}) \quad r ::= 1 \mid \omega \quad (\text{usage/reference counter})$$

where r is a usage marker that says a variable was used either once (denoted 1) or used more than once (denoted ω). Usage has a preorder $\leq$ where $1 \leq \omega$.

We extend the syntax of terms with a value form a representing runtime array references to the heap. In order to account for their type, the syntax of

contexts is extended to include assumptions $a : \mathbf{Array} \ A$ which are treated as a different syntactic category of variables. Additional runtime typing rules for array reference terms a are provided akin to a variable rule (see appendix [28]).

Heaps are defined as follows akin to a context but containing two kinds of ‘allocations’ for variables x and for array references a :

$$H ::= \emptyset \mid H, x \mapsto_r (\Gamma \vdash t : A) \mid H, a \mapsto_r \mathbf{arr} \quad (\text{heaps})$$

In the case of extending the heap with a variable allocation for x , the heap records that x can be used according to r and that it maps to a term t , along with its typing which is only present to aid the metatheory. For brevity, we sometimes write $x \mapsto_r t$ instead of $x \mapsto_r (\Gamma \vdash t : A)$ when the typing is not important. In the case of an array reference a , the heap records the number of references currently held to it, where r is again used (representing either one reference 1 or many ω), and describes the heap-only array representation term $\mathbf{arr}$ pointed to by that reference (whose syntax we introduce later along with the relevant rules).

Multiple reductions are composed from zero or more single-step reductions, with judgments of the form $H \vdash t \Rightarrow H' \vdash t' \mid \Gamma \mid \Delta$ given by two rules capturing empty reduction sequences and extending a sequence at its head:

$$\frac{}{H \vdash t \Rightarrow H \vdash t \mid \emptyset \mid \emptyset} \text{REFL} \quad \frac{\begin{array}{c} H \vdash t_1 \rightsquigarrow H' \vdash t_2 \mid \Gamma_1 \mid \Delta_1 \\ H' \vdash t_2 \Rightarrow H'' \vdash t_3 \mid \Gamma_2 \mid \Delta_2 \end{array}}{H \vdash t_1 \Rightarrow H'' \vdash t_3 \mid \Gamma_1, \Gamma_2 \mid \Delta_1 + \Delta_2} \text{EXT}$$

In the case of EXT the binding contexts are disjoint (since we treat binders as unique in a standard way) but the usage contexts are added as follows:

$$\Delta_1 + \emptyset = \emptyset \quad \Delta_1 + (\Delta_2, x : r) = \begin{cases} (\Delta_1 + \Delta_2), x : r & x \notin \text{dom}(\Delta_1) \\ (\Delta'_1 + \Delta_2), x : \omega & \Delta_1 = \Delta'_1, x : r' \end{cases}$$

i.e., if a variable x appears in both usage contexts then in the resulting context $x : \omega$ since for the purposes of our counting we are interested in counting 0 uses (via absence in Δ) or 1 use or many uses (ω).

Heap model The reduction rules are collected in the appendix [28], but we explain the core rules for the single-step reduction relation here. Unlike a normal small-step semantics, variables have a reduction, with two possibilities:

$$\frac{}{H, x \mapsto_1 t \vdash x \rightsquigarrow H \vdash t \mid \emptyset \mid x : 1} \rightsquigarrow^{\text{VAR}1} \quad \frac{}{H, x \mapsto_\omega t \vdash x \rightsquigarrow H, x \mapsto_\omega t \vdash t \mid \emptyset \mid x : 1} \rightsquigarrow^{\text{VAR}\omega}$$

Both reduce a variable x to the term t which is assigned to x in the heap. In the left rule, we started out with a heap capability of 1 (linear) so after the reduction we remove x from the heap. In the right rule, we have a heap capability of ω (non-linear) so we preserve the assignment to x in the outgoing heap.

β -reduction is then given as follows:

$$\frac{\Gamma \vdash t' : A}{H \vdash (\lambda x. t) t' \rightsquigarrow H, x \mapsto_1 (\Gamma \vdash t' : A) \vdash t \mid x : A \mid \emptyset} \rightsquigarrow^\beta$$

Rather than using a substitution, the body term is the result under a heap extended with x assigned to the (typed) argument term t' . This heap binding is given a resource capability of 1 since functions are linear. In the output, we remember that a linear binding has been opened up in the scope of the term. An inductive rule allows an application to reduce on the left:

$$\frac{H \vdash t_1 \rightsquigarrow H' \vdash t'_1 \mid \Gamma \mid \Delta}{H \vdash t_1 t_2 \rightsquigarrow H' \vdash t'_1 t_2 \mid \Gamma \mid \Delta} \rightsquigarrow_{\text{APP}}$$

We elide the rules for products and unit which follow much the same scheme; one congruence to evaluate the reduction of an elimination form and one to enact a β reduction. For the $!$ modality, this scheme gives us the $!\beta$ rule which creates a non-linear binding of x to the term t_1 :

$$\frac{[\Gamma] \vdash t_1 : A}{H \vdash \text{let } !x = !t_1 \text{ in } t_2 \rightsquigarrow H, x \mapsto_{\omega} ([\Gamma] \vdash t_1 : A) \vdash t_2 \mid x : [A] \mid \emptyset} \rightsquigarrow_{!\beta}$$

The more interesting rules are for the uniqueness aspects of the language. Borrowing $\&$ (which maps a unique value type $*A$ to a non-linear value $!A$) has a congruence rule and a reduction to enact a borrow:

$$\frac{H \vdash t \rightsquigarrow H' \vdash t' \mid \Gamma \mid \Delta}{H \vdash \&t \rightsquigarrow H' \vdash \&t' \mid \Gamma \mid \Delta} \rightsquigarrow_{\&} \quad \frac{\text{dom}(H) \equiv \text{arrRefs}(v)}{H, H' \vdash \&(*v) \rightsquigarrow ([H]_{\omega}, H' \vdash !v \mid \emptyset \mid \emptyset)} \rightsquigarrow_{\&\beta}$$

The action is in the right-hand rule here, where the incoming heap is split into two parts, where H is such that it provides the allocations for all array references in v (enforced by the premise here). The unique value $*v$ is wrapped to be non-linear in the result $!v$ and thus all of its array references are now marked as ‘many’ via $[H]_{\omega}$ which replaces all reference counts with ω , e.g.:

$$H', a \mapsto_{\mathbf{1}} \mathbf{arr} \vdash \&(*a) \rightsquigarrow H', a \mapsto_{\omega} \mathbf{arr} \vdash !a \mid \emptyset \mid \emptyset$$

Thus, borrowing enacts the idea that a reference is no longer unique and may be used many times (and hence now is a non-linear value). Copying then has three reductions; a congruence (elided), a reduction which forces evaluation under the non-linear modality, and a β -reduction to enact copying to a unique value:

$$\frac{H \vdash t \rightsquigarrow H' \vdash t' \mid \Gamma \mid \Delta}{H \vdash \text{copy } !t \text{ as } x \text{ in } t_2 \rightsquigarrow H' \vdash \text{copy } !t' \text{ as } x \text{ in } t_2 \mid \Gamma \mid \Delta} \rightsquigarrow_{\text{copy}!} \\ \frac{I \vdash v : A \quad \text{dom}(H') \equiv \text{arrRefs}(v) \quad (H'', \theta) \equiv \text{copy}(H')}{H, H' \vdash \text{copy } !v \text{ as } x \text{ in } t \rightsquigarrow H, H', H'', x \mapsto_{\mathbf{1}} (I \vdash *\theta(v) : *A) \vdash t \mid x : *A \mid \emptyset} \rightsquigarrow_{\text{copy}\beta}$$

The $\rightsquigarrow_{\text{copy}!}$ rule evaluates under $!$ so that the first term can be reduced to a value v to be copied in the next rule. The $\rightsquigarrow_{\text{copy}\beta}$ rule enacts copying where $\text{dom}(H') \equiv \text{arrRefs}(v)$ marks the part of the heap with array references coming from v . Then $\text{copy}(H')$ copies the arrays in this part of the heap, creating a heap fragment H'' and a renaming θ which maps from the old array references to the

references of the new copies. This renaming is applied to v in the newly bound unique variable x . Thus the value $\theta(v)$ refers to any freshly copied arrays.

Lastly, the semantics of the array primitives uses an array representation on the heap, where $\mathbf{arr}$ is some array object and $\mathbf{arr}[i] = v$ indicates that the i^{th} element is bound to the value v , and we write $a \# H$ for an array reference a which is fresh for heap H :

$$\begin{array}{c} \frac{a \# H}{H \vdash \mathbf{newArray} \, n \rightsquigarrow H, a \mapsto_{\mathbf{1}} \mathbf{arr} \vdash *a \mid \emptyset \mid \emptyset} \\[10pt] \frac{H, a \mapsto_{\mathbf{r}} (\mathbf{arr}[i] = v) \vdash \mathbf{readArray} \, (*a) \, i \rightsquigarrow H, a \mapsto_{\mathbf{r}} (\mathbf{arr}[i] = v) \vdash (v, *a) \mid \emptyset \mid \emptyset} \\[10pt] \frac{H, a \mapsto_{\mathbf{r}} \mathbf{arr} \vdash \mathbf{writeArray} \, (*a) \, i \, v \rightsquigarrow H, a \mapsto_{\mathbf{r}} (\mathbf{arr}[i] = v) \vdash *a \mid \emptyset \mid \emptyset} \\[10pt] \frac{}{H, a \mapsto_{\mathbf{r}} \mathbf{arr} \vdash \mathbf{deleteArray} \, (*a) \rightsquigarrow H \vdash \mathbf{unit} \mid \emptyset \mid \emptyset} \end{array}$$

Thus $\mathbf{newArray}$ creates a fresh array reference a and allocates a new array on the heap with a single reference count. The $\mathbf{readArray}$ and $\mathbf{writeArray}$ primitives work as expected to read and destructively update the array referenced by a , whose reference count is arbitrary but unchanged by the reduction. Lastly $\mathbf{deleteArray}$ deallocates the array. Noticeably, the rules do not enforce uniqueness; but as we see in the next section, well-typed programs preserve uniqueness of references.

3.5 Metatheory

Proofs of all the statements that follow are provided in the appendix [28]. We first establish some key results showing the admissibility of substitution and weakening, which are leveraged in later proofs:

Lemma 1 (Linear substitution). *If $\Gamma' \vdash t' : A$ and $\Gamma, x : A \vdash t : B$ then $\Gamma' + \Gamma \vdash [t'/x]t : B$.*

Lemma 2 (Non-linear substitution). *If $[\Gamma'] \vdash t' : A$ and $\Gamma, x : [A] \vdash t : B$ then $[\Gamma'] + \Gamma \vdash [t'/x]t : B$.*

Lemma 3 (Weakening is admissible). *If $\Gamma \vdash t : A$ then $\Gamma, [\Gamma'] \vdash t : A$.*

Next, the heap model allows us to establish the key properties of well-typed programs respecting linearity and uniqueness restrictions. We first define when a heap is *compatible* with a typing context:

Definition 3 (Heap-context compatibility). A heap H is compatible with a typing context Γ if H contains assignments for every variable in the context and the typing contexts of the terms in the heap are also compatible with the heap. The relation is defined inductively as:

$$\begin{array}{c} \frac{H \bowtie \Gamma}{H, a \mapsto_{\mathbf{r}} \mathbf{arr} \bowtie \Gamma, a : \mathbf{Array} \, A} \text{REF} \quad \frac{H \bowtie (\Gamma_1 + \Gamma_2) \quad \Gamma_2 \vdash t : A \quad x \notin \text{dom}(H)}{(H, x \mapsto_{\mathbf{r}} (\Gamma_2 \vdash t : A)) \bowtie (\Gamma_1, x : A)} \text{LIN} \\[10pt] \frac{}{\emptyset \bowtie \emptyset} \text{EMPTY} \quad \frac{H \bowtie (\Gamma_1 + [\Gamma_2]) \quad [\Gamma_2] \vdash t : A \quad x \notin \text{dom}(H)}{(H, x \mapsto_{\omega} ([\Gamma_2] \vdash t : A)) \bowtie (\Gamma_1, x : [A])} \omega \end{array}$$

Thus, a heap compatible with $\Gamma_1, x : A$ contains an assignment for x marked with a usage annotation r which can be either 1 for linear or ω for non-linear use. Note that non-linear values can be used linearly, as captured by dereliction (the DER typing rule). However, a non-linear assumption must have a heap assignment marked with ω (rule ω), where the dependencies of the assigned term t must all be non-linear in the remaining compatibility judgment on the rest of the heap.

From a heap (and likewise from a typing context) we can also extract usage information. This is useful for focusing on resource usage as follows:

Definition 4 (Usage context extraction). For a context Γ or heap H we can extract usage information denoted $\overline{\Gamma}$ or $\overline{H}$ defined as:

$$\begin{aligned} \overline{\emptyset} &= \emptyset & \overline{(\Gamma, x : [A])} &= \overline{\Gamma}, x : \omega & \overline{(\Gamma, a : A)} &= \overline{\Gamma} & \overline{(\Gamma, x : A)} &= \overline{\Gamma}, x : 1 \\ \overline{\emptyset} &= \emptyset & \overline{(H, x \mapsto_r (\Gamma \vdash t : A))} &= \overline{H}, x : r & \overline{(H, a \mapsto_r t)} &= \overline{H} \end{aligned}$$

We now give the two main theorems about our calculus which give us the properties that linearity is respected (called *conservation*, Theorem 4) and that uniqueness is respected (Theorem 5).

Theorem 4 (Conservation). For a well-typed term $\Gamma \vdash t : A$ and all Γ_0 and H such that $H \bowtie (\Gamma_0 + \Gamma)$ and a reduction $H \vdash t \rightsquigarrow H' \vdash t' \mid \Gamma_1 \mid \Delta$ we have:

$$\exists \Gamma'. \Gamma' \vdash t' : A \quad \wedge \quad H' \bowtie (\Gamma_0 + \Gamma') \quad \wedge \quad (\overline{H'} + \Delta) \sqsubseteq (\overline{H}, \overline{\Gamma_1})$$

The first conjunct is regular type preservation, linked with heap compatibility in the second conjunct. The last conjunct expresses the core of conservation: that resource usage accrued in this reduction, given by Δ , plus remaining resources given in the heap H' are approximated (via $\sqsubseteq$, the pointwise lifting of $\leq$) by the original resources given in the heap H plus the specification of the resources from any variable bindings Γ_1 encountered along the way. The context Γ_0 accounts for bindings not described by Γ , and is key to the inductive proof of this result.

We then establish that all heap references have only one reference to them at the end of execution.

Theorem 5 (Uniqueness). For a well-typed term $\Gamma \vdash t : *A$ and all Γ_0 and H such that $H \bowtie (\Gamma_0 + \Gamma)$ and given a multi-reduction to a value $H \vdash t \Rightarrow H' \vdash *v \mid \Gamma' \mid \Delta$, for all $a \in \text{arrRefs}(v)$ (array references in v) we have:

$$a \mapsto_1 t' \in H \Rightarrow \exists t''. a \mapsto_1 t'' \in H' \quad \wedge \quad a \notin \text{dom}(H) \Rightarrow \exists t''. a \mapsto_1 t'' \in H'$$

i.e., any array references contributing to the final term that are unique in the incoming heap stay unique in the resulting term, and any new array references contributing to the final term are also unique.

Notice that there is a certain duality between the conservation theorem and the uniqueness theorem which mirrors the weak duality between linearity and uniqueness. The statement of conservation is a generalised way to say that if a variable is linear then it will always be used in a linear way, or in other words that

linearity restrictions will always be upheld; conversely, the uniqueness theorem tells us that if a variable is unique then it must always have been used in a unique way, or in other words that it does not have multiple references.

One important point to notice is that the additional rules (borrow and copy) that we include for unique types are in fact trivial cases when it comes to the uniqueness theorem since they can never output a value with a unique type. This makes sense as the idea behind these additional rules is to mediate the interaction between uniqueness and non-uniqueness, and this interaction can only ever go in the direction of producing values that are non-unique.

A sub-result of conservation is type preservation which is complemented by a separate progress result in Theorem 6 to give syntactic type safety:

Theorem 6 (Progress). *Values of the heap model v are given by:*

$$v ::= (t_1, t_2) \mid \text{unit} \mid *t \mid !t \mid \lambda x.t \mid i \mid a \mid p \quad (\text{value terms sub-grammar})$$

where p are partially-applied primitives, e.g., `newArray`, `readArray`, `readArray(*a)`. Given $\Gamma \vdash t : A$, then t is either a value, or if $H \bowtie \Gamma_0 + \Gamma$ there exists a heap H' , term t' , usage context Δ , and context Γ' such that $H \vdash t \rightsquigarrow H' \vdash t' \mid \Gamma' \mid \Delta$.

Finally, we see that the operational semantics, extended to full β -reduction (i.e., all congruences), supports the equational theory:

Theorem 7 (Soundness with respect to the equational theory). *For all t_1, t_2 such that $\Gamma \vdash t_1 : A$ and $\Gamma \vdash t_2 : A$ and $t_1 \equiv t_2$ and given H such that $H \bowtie \Gamma$, there exists a value (irreducible term) v and $\Gamma_1, \Gamma_2, \Delta_1, \Delta_2$ such that there are full β -reductions to the same value*

$$H \vdash t_1 \Rightarrow_\beta H' \vdash v \mid \Gamma_1 \mid \Delta_1 \quad \wedge \quad H \vdash t_2 \Rightarrow_\beta H' \vdash v \mid \Gamma_2 \mid \Delta_2$$

4 Implementation

4.1 Frontend

The implementation of uniqueness types in Granule follows much the same pattern as the logic defined earlier. Granule already possesses a *semiring graded necessity modality*, where for a pre-ordered semiring $(\mathcal{R}, *, 1, +, 0, \sqsubseteq)$, there is a family of types $\{\Box A_r\}_{r \in \mathcal{R}}$. We represent the $!$ from linear logic (and our calculus) via the pre-ordered semiring $\{0, 1, \omega\}$ (none-one-tons [30]) with $!A = \Box A_\omega$.⁷

The semiring is defined with $r + s = r$ if $s = 0$, $r + s = s$ if $r = 0$ and otherwise ω , and $r * 0 = 0 * r = 0$, $r * \omega = \omega * r = \omega$ (for $r \neq 0$), and $r * 1 = 1 * r = r$ with ordering $0 \sqsubseteq \omega$ and $1 \sqsubseteq \omega$. This semiring allows us to represent both linear and non-linear use: variables graded with 1 must be used linearly, with 0 must be discarded, and a grade of ω permits unconstrained use à la linear logic's $!$.

⁷ It may not seem obvious that such a graded modality does exactly represent the behaviour of linear logic's $!$, and in fact capturing the precise behaviour of $!$ does require some additional semiring structure which is present in Granule [22].

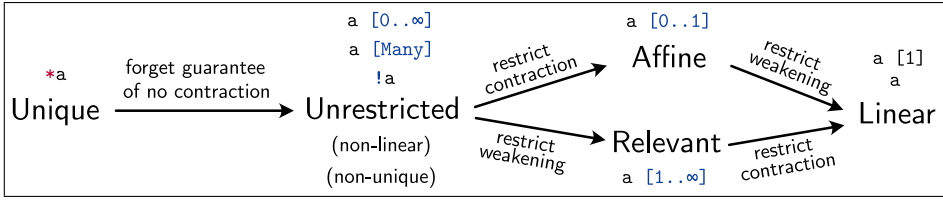


Fig. 2: Relationship between various flavours of substructural types demonstrating how they can all be represented using Granule’s expressive modalities.

(In Granule, $\Box A_\omega$ can be written as the type **A** [Many], but we syntactically alias this to **!A** for simplicity and ease of understanding.)

As in LCU, uniqueness is represented by a new modality, which we call ***** to match the calculus (and so that the syntax of programs involving uniqueness will be familiar to Clean users). The uniqueness modality wraps a value that behaves ‘linearly’ (and so cannot be duplicated or discarded), with the key difference being that we provide primitive functions which allow **!** to act as a relative monad over unique values. The primitives have the following type signatures:

```

1 uniqueReturn : ∀ {a : Type} . *a → !a          -- borrow
2 uniqueBind   : ∀ {a b : Type} . (*a → !b) → !a → !b  -- copy

```

Granule

The `uniqueReturn` function here implements the **BORROW** rule from the calculus (acting as the ‘return’ of the relative monad), and similarly the `uniqueBind` function implements the **COPY** rule (acting as the ‘bind’).

We provide syntactic sugar for both of these primitives for convenience, with syntax designed to evoke the rules from the LCU calculus; `&x` is equivalent to writing `uniqueReturn x`, while `clone t1 as x in t2` is equivalent to writing `uniqueBind (λx → t2) t1`.⁸ A simple example of uniqueness types in action is given below, to demonstrate the idea.

```

1 sip : *Coffee → (Coffee, Awake)
2 sip fresh = let !coffee = &fresh in (keep coffee, drink coffee)

```

Granule

Here, borrowing (`&`) converts the unique `Coffee` value into an unrestricted one, so that it can be duplicated and used twice for the two separate functions. Note however that the uniqueness guarantee is lost in the process, so both of the output values are non-unique (linear, in this case).

Figure 2 illustrates the relationship between uniqueness, linearity and other common forms of substructural typing in the resulting system.

⁸ In the implementation we use ‘clone’ rather than ‘copy’, as the name ‘copy’ is often used elsewhere in Granule, e.g. for the non-linear function which duplicates its input.

We implemented a built-in library for primitive floating point arrays in Granule, matching the interface for arrays of floats that was introduced as an extension to the LCU calculus in Section 3.3, with operations typed as follows:

```

1  newFloatArray   : Int → *FloatArray
2  readFloatArray  : *FloatArray → Int → (Float, *FloatArray)
3  writeFloatArray : *FloatArray → Int → Float → *FloatArray
4  deleteFloatArray : *FloatArray → ()

```

Granule

The `writeFloatArray` primitive updates an array destructively in place since we have a guarantee that no other references exist to the array which has been passed in. In the next section, we use this set of primitives to evaluate the performance of our implementation, by measuring the performance gains from allowing for in-place updates in this fashion. We have another set of *immutable* primitives akin to the above (but written with a suffix `I`) which work with non-unique arrays, e.g. `readFloatArrayI : FloatArray → Int → (Float, FloatArray)`, and thus do not perform mutation.

The following shows an example of `clone`, where a new array is borrowed and a copy of this borrowed `FloatArray` on line 3 is deleted, leaving the original (now immutable) instance of the array unaffected on line 4:

```

1  let x = newFloatArray 10 in
2  let [y] = &x in
3  let [()] = clone [y] as y' in (let () = deleteFloatArray y' in [()])
4  in readFloatArrayI y 10

```

Granule

4.2 Compilation and Evaluation

As part of our implementation of uniqueness types in Granule, as described in Section 4.1, we also implemented a simple compiler that translates programs into Haskell. This compiler preserves the value types, but erases all of Granule’s substructural types (linear, unique, graded, etc.). As a result, we can take advantage of both Granule’s flexible type system and Haskell’s libraries and optimizing compiler. For this paper, all performance results were measured by compiling Granule programs to Haskell, and compiling the resulting Haskell with GHC 9.0.1. The measurements were collected on an ordinary MacBook with a 2 GHz quad-core Intel i5 processor and 16 GB of RAM.

As mentioned in Section 1, one motivation for using uniqueness types is to do the kind of in-place mutation necessary for efficient programming with arrays. To check that our implementation is reasonable, we carried out an evaluation using an array processing benchmark. The benchmark recursively allocates and sums up lists of arrays of various sizes, with the goal of demonstrating the benefits of uniqueness types for arrays in functional programming. Each iteration of the benchmark allocates a list of a thousand arrays, populates the arrays with values, then traverses the list to sum them up. We prepared two versions of this

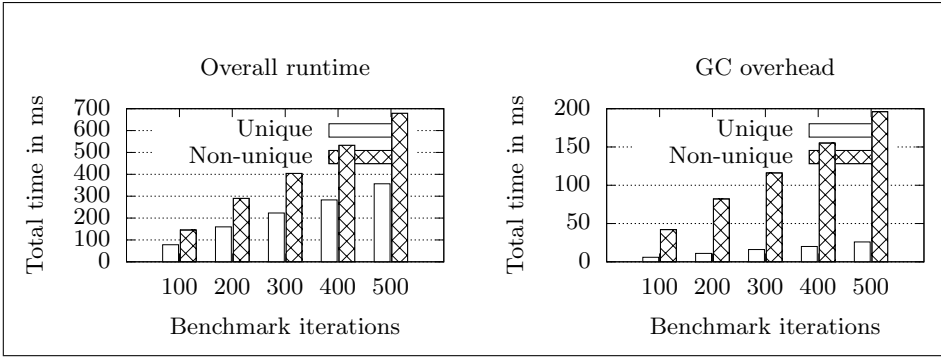


Fig. 3: Performance of mutable vs. immutable arrays in Granule. Lower is better.

benchmark: one with functional in-place updates and manual (safe) deletion of unique arrays, and one with non-unique, immutable, garbage collected arrays and updates via copying. The overall performance of these two benchmarks is shown in Figure 3, with lower bars/numbers representing better performance. The results, while not surprising, do confirm that array-handling is generally more efficient when in-place mutation is allowed. Additionally, in Figure 3, we compared the time spent in garbage collection between the two versions of the benchmark. Because our implementation allocates unique data outside of GHC’s heap, and uniqueness types allow programmers to directly de-allocate objects in memory, the unique version of the benchmark spends significantly less time in garbage collection. For this benchmark, the unique arrays are outside of the garbage collected heap and directly de-allocated, while other incidental objects (closures, lists, and so on) are still handled by the garbage collector.

Of course, this is a somewhat contrived benchmark. Real-world Haskell libraries, for example, typically provide functional high-level interfaces for array manipulation while using *unsafe* code to mutate arrays internally. The popular **vector** library⁹ is one example, and **repa** [25] is another. Additionally, there is significant prior work on improving the efficiency of functional programs operating on arrays (for example, using combinators like **map** and **fold** along with aggressive fusion [13,25,27]), which we will not dwell on. The main point is that, at some stage, arrays must be mutated. Rather than having this happen through unsafe code, or via external C or Fortran, uniqueness types give us a way to do that mutation directly in our functional language, efficiently and safely.

Crucially, in these comparisons, all versions of the programs are implemented in the same language: Granule. With our extensions, the language is expressive enough to encompass a variety of programming approaches. Functional programmers may freely mix and match from a variety of options for data management and manipulation. Object lifetimes may be either manually or automatically managed, and object contents may allow in-place mutation or be immutable.

⁹ <https://hackage.haskell.org/package/vector>

5 Related Work

Uniqueness types are most well known for their appearance in the Clean language [40, 47], where they are used in lieu of monadic computation and for the efficiency gains offered by in-place update. In Clean, computation is based on graph rewriting and reduction; constants such as numbers are graphs, and functions are graph rewriting formulas. This gives the type system a rather different feel to those offered by more recent functional programming languages, and makes it more difficult to capture the benefits of Clean-style uniqueness in a modern setting, hence the value in our pursuit of this goal.

Some theoretical groundwork for Clean’s uniqueness types has certainly been developed over the years, particularly in works by de Vries among others [55, 56]; these papers aim to clarify the distinction between Clean’s type system and systems based on the λ -calculus. Further work makes headway on the problem of distinguishing uniqueness from other substructural systems [53, 54]. This follows a similar theoretical approach to the one demonstrated in our paper; such ideas for limited settings inspired the groundwork for our system, which is more general and has a practical implementation.

Other languages (old and new) featuring uniqueness types include Single-Assignment C [45], Mercury [48] and Cogent [35].

Ownership was first developed as a framework for understanding aliasing in object-oriented languages [34], and is intended to give a high-level structural view of objects and references in much the same way that powerful type systems give a high-level structural view of data. Ownership is now most familiar due to being pervasive in the Rust programming language, for which multiple formalisations have been attempted; RustBelt [24] gives a lower-level encoding of Rust intended for formal verification while Oxide [64] is a higher-level encoding designed for more theoretical work, among others [39]. Extending these ideas to other languages is an active area of research; RefinedC [44] is one example.

Regions have been used over the years in the context of effect systems [23, 26]. One of the primary motivations of research into region types was their application in *region-based memory management* [51], which aimed to bring some of the benefits of traditional stack-based memory management to higher-order functional languages. Regions divide values based on their lifetimes, so a system with region types can safely allocate and de-allocate memory for values based on region type information, eliminating the need for garbage collection.

Early on, regions were restricted to have LIFO (last-in, first-out) lifetimes which followed the block structure of a language, but later work relaxed this constraint using uniqueness (see: static capabilities [63] and Cyclone [21]); a unique reference to a region ensures there are no aliases to the region, and that it can therefore be promptly de-allocated. Additionally, regions themselves act as a way to control aliasing, and can be thought of as equivalence classes for a “may alias” relation—in other words, values which do not share a region may not alias with one another, and so if a value does not share a region with anything else then it may be safely mutated in place.

Work on Cyclone [14] demonstrated the relationship between regions and unique pointers, observing that “unique pointers are essentially lightweight, dynamic regions that hold exactly one object.” Beyond that, Rust’s lifetimes are heavily based on regions, and there exists an extension of ML called Affe [43] which aims to support both linearity and borrowing using regions.

Capabilities are tokens that a function must possess in order to be able to access a particular location in memory. Capabilities are linear, and cannot be duplicated or discarded, in order to prevent them from being forged [17]. Implementations exist for various object-oriented languages such as Java [2] and Scala [19]; more functional languages taking inspiration from the idea of capabilities also exist [33, 41]. Recent work on linear constraints for Haskell [49], which hopes to allow for something similar to borrowing within the framework of linear Haskell, also descends from work on capabilities. Ambient capabilities can also be internalised as a comonad to capture purity within an impure language [12].

6 Future Work

Ownership via fractional permissions Though Granule can now represent values with both linear and unique types, the language allows for much more fine-grained analysis of resourceful data via *grading*. For instance, we can replay our earlier non-linearity example but with some extra information in the types:

```
1 accurate : Cake [2] → (Happy, Cake)
2 accurate [cake] = let extra = have cake in (eat cake, extra)
```

Granule

Instead of an infinite amount of cake we specify that we have exactly two cakes; the cake on the right-hand side must be linear as we only have one usage remaining. If we used the input three times we would receive a type error.

Given that we can move beyond the simple binary view of linear and non-linear, one might suspect that we could track the quantity of existing references to a value more accurately than just unique or non-unique. We propose taking inspiration here from Boyland’s notion of fractional permissions [8].

The purpose of fractional permissions is to allow multiple readers to access the same resource without losing the ability to later gain unique write access. A “permission” can be split up, allowing read-only access to multiple consumers, and then later recombined (while ensuring no other permissions still exist).

To relate these with our calculus, let us hypothesise that $*_1 P$ is a ‘complete’ unique value that we can read from or write to, and that we can split up arbitrarily into ‘fractionally’ unique values $*_n P$ where $0 < n \leq 1$, as follows:

$$*_n P \otimes *_m P \longleftrightarrow *_{n+m} P$$

As with fractional permissions, fractional values must only be used for behaviour that does not involve mutation, because whilst a value is only fractionally unique we cannot guarantee that other references do not exist. We should only regain this ability if we recombine the guarantees into a complete $*_1 P$.

This model closely resembles ownership as in Rust [24] – we can think of a value of type $*_n P$ for $n < 1$ as being equivalent to a Rust-style $\&P$ which is a borrowed value that we cannot mutate.¹⁰ When a value has been borrowed the original value cannot be written to until we are finished with the borrows, much like we would need to collect all the fractionally unique values back together to get back to our original unique $*_1 P$. Being able to more closely model Rust’s powerful ownership system would make this a fruitful avenue for future research.

Linear Haskell Granule’s linear basis and assortment of modalities allows for a particularly natural embedding of the LCU calculus, but this does not preclude the theory of this paper from being applied in other contexts. One particularly valuable setting to consider would be Haskell, which as of GHC 9 already has linear types based on an underlying graded system called $\lambda^q_{\downarrow}$.

Haskell’s graded representation of linearity involves function types $(a \text{ } \%r \rightarrow b)$ which have a multiplicity annotation r ; at present, this can be either `'One` (linear) or `'Many` (unrestricted). But $\lambda^q_{\downarrow}$ is designed to be extensible, and the possibility of introducing additional multiplicities is welcomed [7, 49].

The original paper on linear Haskell [7] mentions that “linear types are conceptually simpler than uniqueness type systems, giving a clearer path to implementation in GHC”, and also that “functional languages have more use for fusion than in-place update”. Our clarification of the relationship between linearity and uniqueness demonstrates that not only are uniqueness types no more complex conceptually than linear ones, they can comfortably sit alongside one another in a single calculus; our evaluation demonstrates that while linearity is certainly useful, there are still further practical benefits to be gained from introducing uniqueness into a language with linear types. Perhaps these contributions will begin to forge a path towards a future for Haskell where linear types and uniqueness types can both be leveraged for their respective strengths.

Adjoint models Benton’s linear/non-linear (LNL) logic [6] consists of two fragments: intuitionistic (non-linear) logic $\Phi \vdash_{\mathcal{I}} X$ and a mixed fragment of intuitionistic linear logic with non-linear hypotheses $\Phi, \Gamma \vdash_{\mathcal{L}} A$. These two fragments are connected by a pair of modalities $\text{Lin}(X)$ and $\text{Mny}(A)$, which form an adjunction; the $!$ modality can then be recovered by $!A = \text{Lin}(\text{Mny}(A))$.

Breaking the $!$ modality into two and allowing linear logic to be mixed with non-linear logic has been a valuable endeavour, and so a natural question is whether it is possible to build an LNL-style adjoint model for our unified LCU calculus. It seems plausible that building an adjoint model for just uniqueness logic would not be too difficult; this would be very similar to the LNL model but with the adjunction moving in the opposite direction, and the monadic modality $\circ$ from uniqueness logic could be represented in much the same way that the comonadic $!$ can be recovered in LNL.

¹⁰ Rust also includes *mutable* borrows, which allow the borrower to both read from and write to their borrowed reference. These are a much closer analogue to our current non-fractional calculus, since mutable borrows must be unique.

An adjoint model for the full LCU calculus would be more interesting. This would most likely involve three fragments, two of which would be symmetric monoidal categories (for unique and linear values) and one of which would be a Cartesian closed category (for unrestricted values), with two adjunctions allowing values to flow from unique to unrestricted to linear as we might hope.

Ordered and dependent types As expressive as Granule’s type system may be, there are opportunities for enforcing stronger properties on programs elsewhere in the landscape of type theories. One possibility is that in addition to restricting contraction and weakening, it is also possible to restrict *exchange*, giving ordered type theories which correspond to noncommutative logic.

Such systems can be used to model stack-based memory allocation (as opposed to heap-based), since without exchange an object may only be used when it is at the top of the stack [10,62]. But much like linearity, these systems restrict the use of exchange in the *future*; is there an equivalent of uniqueness for ordered types which guarantees that exchange has never been applied in the *past*, and could this be useful for tracking references on the stack?

Another possibility is to bring uniqueness into the realm of dependent types. Recent work on graded modal dependent type theory (GRTT) [31] allows for capturing requirements on variable usage at both the type and computation levels; grades come in pairs, where the first component is the computation-level grading and the second component is the type-level grading. Strictly linear usage in types is rare – is there value in being able to represent uniqueness here?

7 Conclusion

Linearity and uniqueness are both well-studied concepts with similar substructural foundations, but differing benefits; linearity enables the careful management of resourceful data, while uniqueness offers the possibility of safe in-place updates. By formalising the relationship between these two ideas, building on two distinct bodies of literature, we have shown that there is value in having both linear and unique types in the same type system. This could be a first step on the road towards properly understanding the relationships between more advanced substructural type systems, such as the fine-grained resource tracking of Granule and Idris and the complex memory management provided by Rust.

Moreover, we implemented this system in the graded modal setting of the Granule language and provided benchmarks to demonstrate the efficiency gains that can be accessed via adding uniqueness to a language that already has a linear basis. The opportunities to incorporate uniqueness types into languages outside of Granule are apparent, and this paper offers both a theoretical underpinning for uniqueness as it relates to linearity as well as clear validation of the performance benefits that a system which unifies linearity and uniqueness can offer.

Acknowledgments This work was supported by an EPSRC Doctoral Training Award (Marshall) and EPSRC grant EP/T013516/1 (*Verifying Resource-like Data Use in Programs via Types*).

References

1. Ahrens, B., Spadotti, R.: Terminal semantics for codata types in intensional Martin-Löf type theory. In: Herbelin, H., Letouzey, P., Sozeau, M. (eds.) 20th International Conference on Types for Proofs and Programs (TYPES 2014). Leibniz International Proceedings in Informatics (LIPIcs), vol. 39, pp. 1–26. Schloss Dagstuhl, Germany (Oct 2015). <https://doi.org/10.4230/LIPIcs.TYPES.2014.1>
2. Aldrich, J., Kostadinov, V., Chambers, C.: Alias annotations for program understanding. In: Proceedings of the 17th ACM SIGPLAN Conference on Object-Oriented Programming, Systems, Languages, and Applications. p. 311–330. OOPSLA '02, Association for Computing Machinery, New York, NY, USA (2002). <https://doi.org/10.1145/582419.582448>
3. Altenkirch, T., Chapman, J., Uustalu, T.: Monads need not be endofunctors. Logical Methods in Computer Science **11**(1) (Mar 2015). [https://doi.org/10.2168/lmcs-11\(1:3\)2015](https://doi.org/10.2168/lmcs-11(1:3)2015)
4. Baker, H.G.: Lively linear Lisp: “look ma, no garbage!”. SIGPLAN Not. **27**(8), 89–98 (Aug 1992). <https://doi.org/10.1145/142137.142162>
5. Barendsen, E., Smetsers, S.: Conventional and uniqueness typing in graph rewrite systems. In: International Conference on Foundations of Software Technology and Theoretical Computer Science. pp. 41–51. Springer (1993)
6. Benton, P.N.: A mixed linear and non-linear logic: Proofs, terms and models. In: Pacholski, L., Tiuryn, J. (eds.) Computer Science Logic. pp. 121–135. Springer Berlin Heidelberg, Berlin, Heidelberg (1995)
7. Bernardy, J.P., Boespflug, M., Newton, R.R., Peyton Jones, S., Spiwack, A.: Linear Haskell: Practical linearity in a higher-order polymorphic language. Proc. ACM Program. Lang. **2**(POPL) (Dec 2017). <https://doi.org/10.1145/3158093>
8. Boyland, J.: Checking interference with fractional permissions. In: Cousot, R. (ed.) Static Analysis. pp. 55–72. Springer Berlin Heidelberg, Berlin, Heidelberg (2003)
9. Brady, E.: Idris 2: Quantitative Type Theory in Practice. In: Møller, A., Sridharan, M. (eds.) 35th European Conference on Object-Oriented Programming (ECOOP 2021). Leibniz International Proceedings in Informatics (LIPIcs), vol. 194, pp. 9:1–9:26. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2021). <https://doi.org/10.4230/LIPIcs.ECOOP.2021.9>, <https://drops.dagstuhl.de/opus/volltexte/2021/14052>
10. Bryant, A.B., Eades, H.D.: The graded Lambek calculus. In: 2020 Joint Workshop on Linearity & TLLA: The 6th Workshop on Linearity and the 4th Workshop on Trends in Linear Logic and Applications (2020), https://www.cs.unibo.it/~dallago/TLLALINEARITY2020/The_Graded_Lambek_Calculus.pdf
11. Choudhury, P., Eades III, H., Eisenberg, R.A., Weirich, S.: A graded dependent type system with a usage-aware semantics. Proceedings of the ACM on Programming Languages **5**(POPL), 1–32 (2021)
12. Choudhury, V., Krishnaswami, N.: Recovering purity with comonads and capabilities. Proc. ACM Program. Lang. **4**(ICFP), 111:1–111:28 (2020). <https://doi.org/10.1145/3408993>
13. Coutts, D., Leshchinskiy, R., Stewart, D.: Stream fusion: From lists to streams to nothing at all. In: Proceedings of the 12th ACM SIGPLAN International Conference on Functional Programming. p. 315–326. ICFP '07, Association for Computing Machinery, New York, NY, USA (2007). <https://doi.org/10.1145/1291151.1291199>

14. Fluet, M., Morrisett, G., Ahmed, A.: Linear regions are all you need. In: Sestoft, P. (ed.) *Programming Languages and Systems*. pp. 7–21. Springer Berlin Heidelberg, Berlin, Heidelberg (2006)
15. Girard, J.Y.: Linear logic. *Theoretical Computer Science* **50**(1), 1 – 101 (1987). [https://doi.org/10.1016/0304-3975\(87\)90045-4](https://doi.org/10.1016/0304-3975(87)90045-4), <http://www.sciencedirect.com/science/article/pii/0304397587900454>
16. Girard, J.Y., Scedrov, A., Scott, P.J.: Bounded linear logic: a modular approach to polynomial-time computability. *Theoretical computer science* **97**(1), 1–66 (1992)
17. Gordon, C.S.: Designing with Static Capabilities and Effects: Use, Mention, and Invariants (Pearl). In: Hirschfeld, R., Pape, T. (eds.) *34th European Conference on Object-Oriented Programming (ECOOP 2020)*. Leibniz International Proceedings in Informatics (LIPIcs), vol. 166, pp. 10:1–10:25. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany (2020). <https://doi.org/10.4230/LIPIcs.ECOOP.2020.10>, <https://drops.dagstuhl.de/opus/volltexte/2020/13167>
18. Guzman, J., Hudak, P.: Single-threaded polymorphic lambda calculus. In: [1990] *Proceedings. Fifth Annual IEEE Symposium on Logic in Computer Science*. pp. 333–343 (1990). <https://doi.org/10.1109/LICS.1990.113759>
19. Haller, P., Odersky, M.: Capabilities for uniqueness and borrowing. In: D’Hondt, T. (ed.) *ECOOP 2010 – Object-Oriented Programming*. pp. 354–378. Springer Berlin Heidelberg, Berlin, Heidelberg (2010)
20. Harrington, D.: Uniqueness logic. *Theoretical Computer Science* **354**(1), 24–41 (2006)
21. Hicks, M., Morrisett, G., Grossman, D., Jim, T.: Experience with safe manual memory-management in Cyclone. In: *Proceedings of the 4th International Symposium on Memory Management*. p. 73–84. ISMM ’04, Association for Computing Machinery, New York, NY, USA (2004). <https://doi.org/10.1145/1029873.1029883>
22. Hughes, J., Marshall, D., Wood, J., Orchard, D.: Linear Exponentials as Graded Modal Types. In: *5th International Workshop on Trends in Linear Logic and Applications (TLA 2021)*. Rome (virtual), Italy (Jun 2021), <https://hal-lirmm.ccsd.cnrs.fr/lirmm-03271465>
23. Jouvelot, P., Gifford, D.: Algebraic reconstruction of types and effects. In: *Proceedings of the 18th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. p. 303–310. POPL ’91, Association for Computing Machinery, New York, NY, USA (1991). <https://doi.org/10.1145/99583.99623>
24. Jung, R., Jourdan, J.H., Krebbers, R., Dreyer, D.: RustBelt: Securing the foundations of the Rust programming language. *Proc. ACM Program. Lang.* **2**(POPL) (Dec 2017). <https://doi.org/10.1145/3158154>
25. Keller, G., Chakravarty, M.M., Leshchinskiy, R., Peyton Jones, S., Lippmeier, B.: Regular, Shape-Polymorphic, Parallel Arrays in Haskell. In: *Proceedings of the 15th ACM SIGPLAN International Conference on Functional Programming*. p. 261–272. ICFP ’10, Association for Computing Machinery, New York, NY, USA (2010). <https://doi.org/10.1145/1863543.1863582>
26. Lucassen, J.M., Gifford, D.K.: Polymorphic effect systems. In: *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. p. 47–57. POPL ’88, Association for Computing Machinery, New York, NY, USA (1988). <https://doi.org/10.1145/73560.73564>
27. Mainland, G., Leshchinskiy, R., Peyton Jones, S.: Exploiting vector instructions with generalized stream fusion. In: *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming*. p. 37–48. ICFP

- '13, Association for Computing Machinery, New York, NY, USA (2013). <https://doi.org/10.1145/2500365.2500601>
28. Marshall, D., Vollmer, M., Orchard, D.: Linearity and Uniqueness: An Entente Cordiale (Appendix). <https://doi.org/10.5281/zenodo.5919193>
29. Marshall, D., Vollmer, M., Orchard, D.: Linearity and Uniqueness: An Entente Cordiale (Artifact). <https://doi.org/10.5281/zenodo.5919209>
30. McBride, C.: I Got Plenty o' Nuttin'. In: A List of Successes That Can Change the World - Essays Dedicated to Philip Wadler on the Occasion of His 60th Birthday. pp. 207–233 (2016). https://doi.org/10.1007/978-3-319-30936-1_12
31. Moon, B., Eades III, H., Orchard, D.: Graded modal dependent type theory. In: Yoshida, N. (ed.) Programming Languages and Systems. pp. 462–490. Springer International Publishing, Cham (2021)
32. Morris, J.G.: The Best of Both Worlds: Linear Functional Programming without Compromise. In: Proceedings of the 21st ACM SIGPLAN International Conference on Functional Programming. p. 448–461. ICFP 2016, Association for Computing Machinery, New York, NY, USA (2016). <https://doi.org/10.1145/2951913.2951925>
33. Morrisett, G., Ahmed, A., Fluet, M.: L³: A linear language with locations. In: Urzyczyn, P. (ed.) Typed Lambda Calculi and Applications. pp. 293–307. Springer Berlin Heidelberg, Berlin, Heidelberg (2005)
34. Mycroft, A., Voigt, J.: Notions of aliasing and ownership. In: Clarke, D., Noble, J., Wrigstad, T. (eds.) Aliasing in Object-Oriented Programming. Types, Analysis and Verification, Lecture Notes in Computer Science, vol. 7850, pp. 59–83. Springer (2013). https://doi.org/10.1007/978-3-642-36946-9_4
35. O'Connor, L., Chen, Z., Rizkallah, C., Jackson, V., Amani, S., Klein, G., Murray, T., Sewell, T., Keller, G.: Cogent: Uniqueness types and certified compilation. J. Funct. Program. (2021)
36. Orchard, D., Liepelt, V.B., Eades III, H.: Quantitative Program Reasoning with Graded Modal Types. Proc. ACM Program. Lang. **3**(ICFP) (Jul 2019). <https://doi.org/10.1145/3341714>
37. Orchard, D., Mycroft, A.: Categorical programming for data types with restricted parametricity. Unpublished note. (2012), <https://www.cs.kent.ac.uk/people/staff/dao7/drafts/tfp-structures-orchard12.pdf>
38. Paykin, J., Zdancewic, S.: The linearity monad. In: Proceedings of the 10th ACM SIGPLAN International Symposium on Haskell. p. 117–132. Haskell 2017, Association for Computing Machinery, New York, NY, USA (2017). <https://doi.org/10.1145/3122955.3122965>
39. Pearce, D.J.: A lightweight formalism for reference lifetimes and borrowing in Rust. ACM Trans. Program. Lang. Syst. **43**(1) (Apr 2021). <https://doi.org/10.1145/3443420>
40. Plasmeijer, R., van Eekelen, M., van Groningen, J.: Clean Version 2.2 Language Report (2011), <https://clean.cs.ru.nl/download/doc/CleanLangRep.2.2.pdf>
41. Pottier, F., Protzenko, J.: Programming with permissions in Mezzo. In: Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming. p. 173–184. ICFP '13, Association for Computing Machinery, New York, NY, USA (2013). <https://doi.org/10.1145/2500365.2500598>
42. Prawitz, D.: Natural Deduction: A proof-theoretical study. Stockholm Studies in Philosophy. Almqvist & Wiksell, Stockholm **3** (1965)
43. Radanne, G., Saffrich, H., Thiemann, P.: Kindly bent to free us. Proc. ACM Program. Lang. **4**(ICFP) (Aug 2020). <https://doi.org/10.1145/3408985>

44. Sammler, M., Lepigre, R., Krebbers, R., Memarian, K., Dreyer, D., Garg, D.: *RefinedC: Automating the foundational verification of C code with refined ownership types*. *Proc. ACM Program. Lang.* (Jun 2021)
45. Scholz, S.B.: *Single Assignment C – Entwurf und Implementierung einer funktionalen C-Variante mit spezieller Unterstützung shape-invarianter Array-Operationen*. Ph.D. thesis, Institut für Informatik und Praktische Mathematik, Christian-Albrechts-Universität, Kiel, Germany (1996), https://www.sac-home.org/_media/publications/pdf:sac-design-sbs-phd-96.pdf, Shaker Verlag, Aachen, 1997
46. Selinger, P., Valiron, B.: *Quantum Lambda Calculus*, p. 135–172. Cambridge University Press (2009). <https://doi.org/10.1017/CBO9781139193313.005>
47. Smetsters, S., Barendsen, E., van Eekelen, M., Plasmeijer, R.: *Guaranteeing safe destructive updates through a type system with uniqueness information for graphs*. In: Schneider, H.J., Ehrig, H. (eds.) *Graph Transformations in Computer Science*. pp. 358–379. Springer Berlin Heidelberg, Berlin, Heidelberg (1994). https://doi.org/10.1007/3-540-57787-4_23
48. Somogyi, Z., Henderson, F., Conway, T.: *The execution algorithm of Mercury, an efficient purely declarative logic programming language*. *The Journal of Logic Programming* **29**(1), 17–64 (1996). [https://doi.org/10.1016/S0743-1066\(96\)00068-4](https://doi.org/10.1016/S0743-1066(96)00068-4), <https://www.sciencedirect.com/science/article/pii/S0743106696000684>, *high-Performance Implementations of Logic Programming Systems*
49. Spiwack, A., Kiss, C., Bernardy, J.P., Wu, N., Eisenberg, R.: *Linear constraints* (2021), <https://arxiv.org/abs/2103.06127>
50. Terui, K.: *Light affine lambda calculus and polytime strong normalization*. In: *LICS '01*. pp. 209–220. IEEE Computer Society (2001)
51. Tofte, M., Birkedal, L., Elsmann, M., Hallenberg, N.: *A Retrospective on Region-Based Memory Management*. *Higher-Order and Symbolic Computation* **17**(3), 245–265 (Sep 2004). <https://doi.org/10.1023/B:LISP.0000029446.78563.a4>
52. Tov, J.A., Pucella, R.: *Practical affine types*. In: *Proceedings of the 38th Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*. p. 447–458. POPL '11, Association for Computing Machinery, New York, NY, USA (2011). <https://doi.org/10.1145/1926385.1926436>
53. de Vries, E.: *Modelling unique and affine typing using polymorphism*. In: *Essays Dedicated to Rinus Plasmeijer on the Occasion of His 61st Birthday on The Beauty of Functional Code - Volume 8106*. p. 181–192. Springer-Verlag, Berlin, Heidelberg (2013). https://doi.org/10.1007/978-3-642-40355-2_13
54. de Vries, E., Francalanza, A., Hennessy, M.: *Uniqueness typing for resource management in message-passing concurrency*. In: Florido, M., Mackie, I. (eds.) *Proceedings First International Workshop on Linearity, LINEARITY 2009, Coimbra, Portugal, 12th September 2009*. EPTCS, vol. 22, pp. 26–37 (2009). <https://doi.org/10.4204/EPTCS.22.3>
55. de Vries, E., Plasmeijer, R., Abrahamson, D.M.: *Uniqueness typing redefined*. In: *Proceedings of the 18th International Conference on Implementation and Application of Functional Languages*. p. 181–198. IFL'06, Springer-Verlag, Berlin, Heidelberg (2006)
56. de Vries, E., Plasmeijer, R., Abrahamson, D.M.: *Uniqueness typing simplified*. In: Chitil, O., Horváth, Z., Zsók, V. (eds.) *Implementation and Application of Functional Languages*. pp. 201–218. Springer Berlin Heidelberg, Berlin, Heidelberg (2008)

57. Wadler, P.: Linear Types Can Change the World! In: Broy, M., Jones, C.B. (eds.) *Programming Concepts and Methods: Proceedings of the IFIP Working Group 2.2/2.3 Working Conference on Programming Concepts and Methods, Sea of Galilee, Israel, 2–5 April, 1990*. pp. 561–581. North-Holland, Amsterdam (1990), <https://homepages.inf.ed.ac.uk/wadler/topics/linear-logic.html#linear-types>
58. Wadler, P.: Is there a use for linear logic? In: *Proceedings of the 1991 ACM SIGPLAN Symposium on Partial Evaluation and Semantics-Based Program Manipulation*. p. 255–273. PEPM '91, Association for Computing Machinery, New York, NY, USA (1991). <https://doi.org/10.1145/115865.115894>
59. Wadler, P.: There's no substitute for linear logic. In: *8th International Workshop on the Mathematical Foundations of Programming Semantics* (1992)
60. Wadler, P.: A syntax for linear logic. In: *Mathematical Foundations of Programming Semantics, 9th International Conference, New Orleans, LA, USA, April 7–10, 1993, Proceedings*. pp. 513–529 (1993). https://doi.org/10.1007/3-540-58027-1_24
61. Wadler, P.: A taste of linear logic. In: Borzyszkowski, A.M., Sokolowski, S. (eds.) *Mathematical Foundations of Computer Science 1993*. pp. 185–210. Springer Berlin Heidelberg, Berlin, Heidelberg (1993)
62. Walker, D.: Substructural type systems. *Advanced Topics in Types and Programming Languages* pp. 3–44 (2005)
63. Walker, D., Crary, K., Morrisett, G.: Typed memory management via static capabilities. *ACM Trans. Program. Lang. Syst.* **22**(4), 701–771 (jul 2000). <https://doi.org/10.1145/363911.363923>
64. Weiss, A., Gierczak, O., Patterson, D., Ahmed, A.: Oxide: The essence of Rust (2021), <https://arxiv.org/abs/1903.00982>
65. Zhu, D., Xi, H.: Safe programming with pointers through stateful views. In: Hermenegildo, M.V., Cabeza, D. (eds.) *Practical Aspects of Declarative Languages*. pp. 83–97. Springer Berlin Heidelberg, Berlin, Heidelberg (2005)

Open Access This chapter is licensed under the terms of the Creative Commons Attribution 4.0 International License (<http://creativecommons.org/licenses/by/4.0/>), which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons license and indicate if changes were made.

The images or other third party material in this chapter are included in the chapter's Creative Commons license, unless indicated otherwise in a credit line to the material. If material is not included in the chapter's Creative Commons license and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder.

